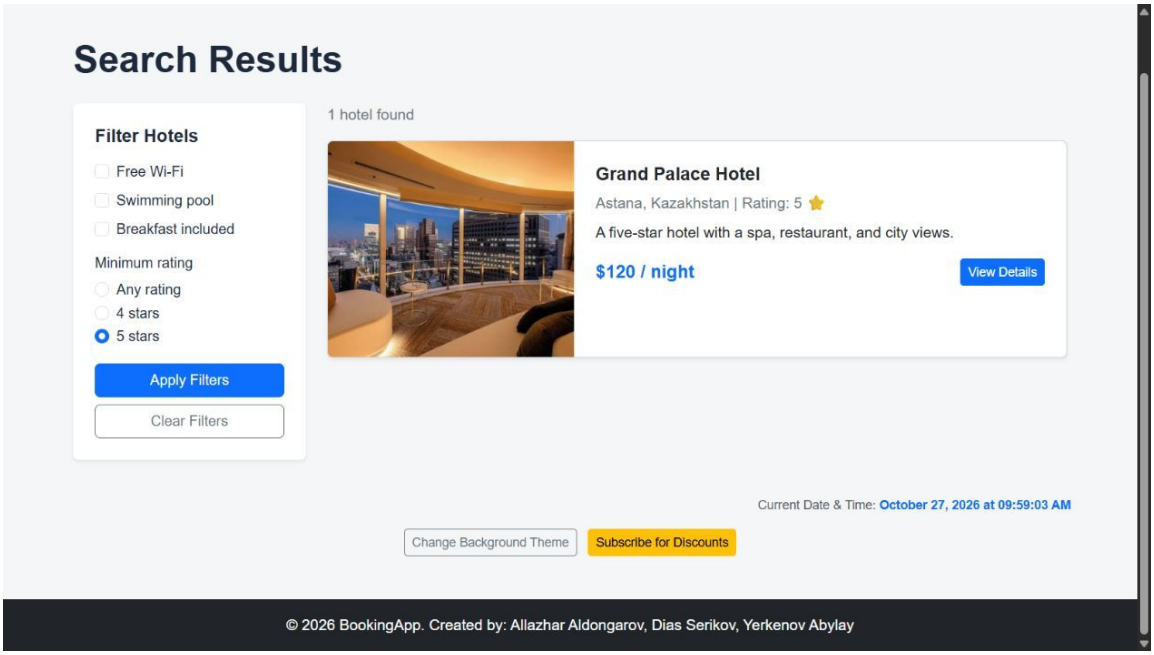
Assignment 5

Team: BookingApp
Group: SE-2513
Members: Allazhar Aldongarov, Dias Serikov, Yerkenov Abylay

We continued our website from Assignment 4. The goal was to add JavaScript features and make the pages respond to user actions.

1. Hotel filters

We put the hotels in an array of objects. The renderHotels function creates cards with forEach and puts them in the results area. When Apply Filters is clicked, switch chooses the rating rule and filter checks the amenities. Then the cards and hotel count update without a reload. Clear Filters shows all three hotels again.



Search results after choosing five stars.

js/script.js

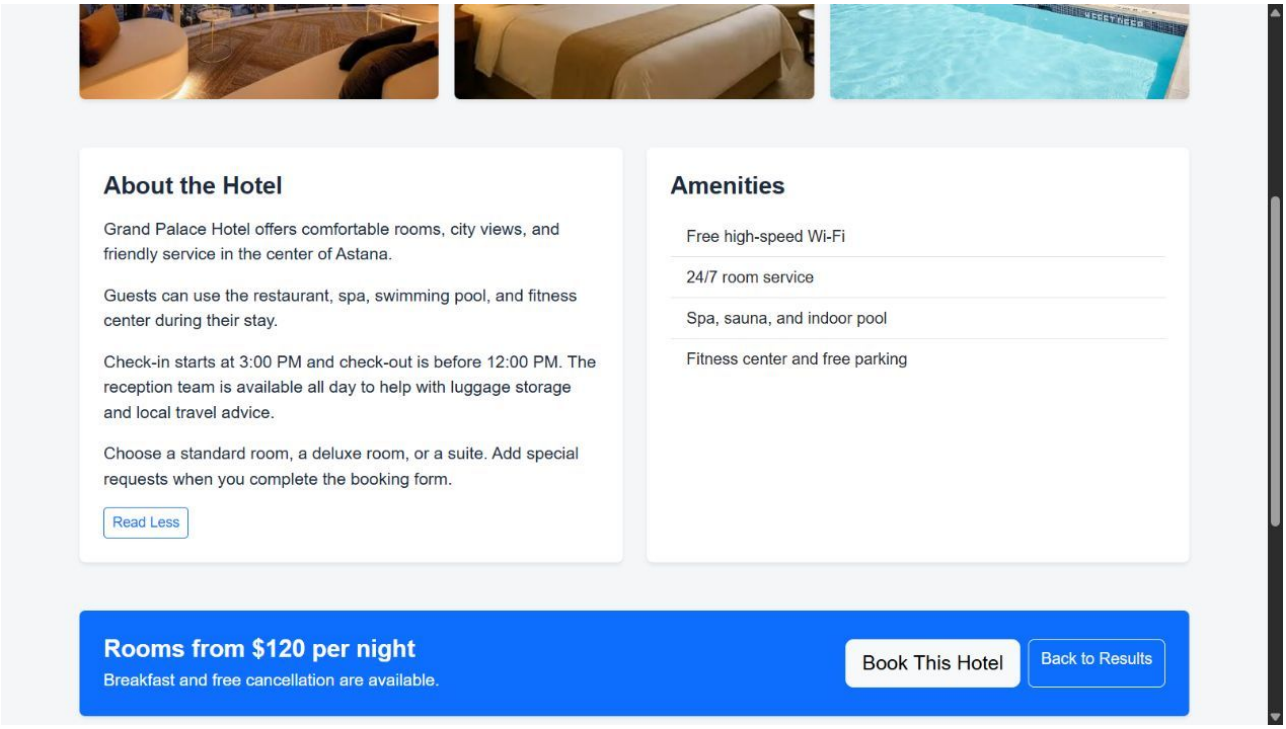
```
165             switch (rating) {
166                 case '4':
167                     filteredHotels = hotelsDatabase.filter(hotel => hotel.rating >= 4);
168                     break;
169                 case '5':
170                     filteredHotels = hotelsDatabase.filter(hotel => hotel.rating === 5);
171                     break;
172                 default:
173                     filteredHotels = hotelsDatabase;
174             }
175             filteredHotels = filteredHotels.filter(function (hotel) {
176                 return (!needsWifi || hotel.wifi) &&
177                     (!needsPool || hotel.pool) &&
178                     (!needsBreakfast || hotel.breakfast);
179             });
180             renderHotels(filteredHotels);
181         });
182         filterForm.addEventListener('reset', function () {
```

The switch statement and filter callbacks in js/script.js.

filter and forEach are higher-order functions because they take another function as an argument.

2. Read More and background

We selected the Read More button and the extra description with getElementById. The click handler changes style.display to show or hide the text. It also changes the button text and aria-expanded.



Hotel Details after clicking Read More.

js/booking.js

```
const readMoreButton = document.getElementById('hotel-read-more');

if (readMoreButton) {
    const moreInformation = document.getElementById('hotel-more');

    readMoreButton.addEventListener('click', function () {
        const isOpen = readMoreButton.getAttribute('aria-expanded') === 'true';
        moreInformation.style.display = isOpen ? 'none' : 'block';
        moreInformation.setAttribute('aria-hidden', String(isOpen));
        readMoreButton.setAttribute('aria-expanded', String(!isOpen));
        readMoreButton.textContent = isOpen ? 'Read More' : 'Read Less';
    });
}

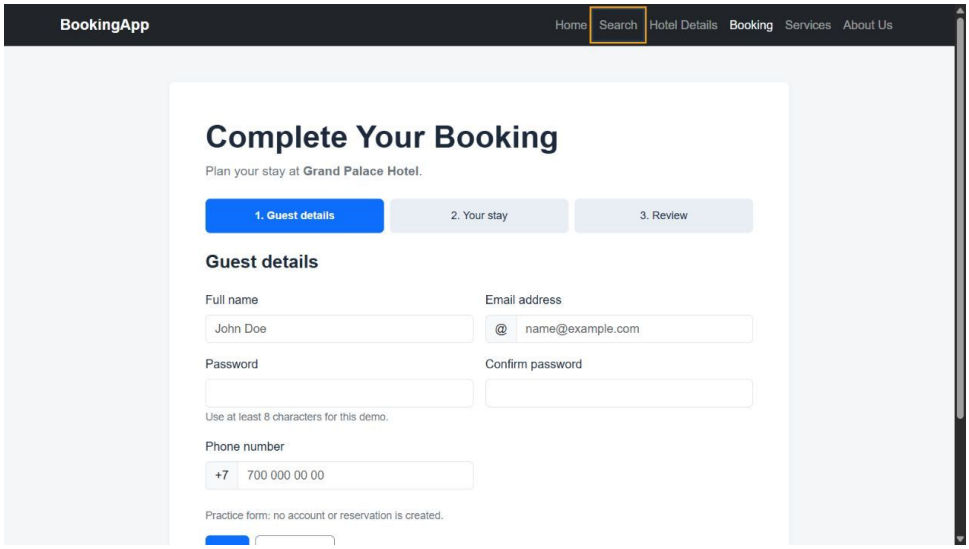
// The identifier from Search chooses the same hotel on both pages.
const selectedHotel = getSelectedHotel();
document.querySelectorAll('a[href="booking.html"]').forEach(function (link) {
```

The Read More handler in js/booking.js.

On Home and Search, Change Background Theme moves to the next colour in an array. JavaScript sets document.body.style.backgroundColor to that colour. These buttons change text, attributes and styles.

3. Keyboard navigation and sound

We used a for loop to add a keydown listener to each menu link. First, the user reaches the menu with Tab. The arrow keys move focus between links, and Enter opens the focused page.



A navigation link with keyboard focus.

js/booking.js

```
250
251     for (let index = 0; index < menuLinks.length; index++) {
252         menuLinks[index].addEventListener('keydown', function (event) {
253             let nextIndex = index;
254
255             switch (event.key) {
256                 case 'ArrowRight':
257                 case 'ArrowDown':
258                     nextIndex = (index + 1) % menuLinks.length;
259                     break;
260                 case 'ArrowLeft':
261                 case 'ArrowUp':
262                     nextIndex = (index - 1 + menuLinks.length) % menuLinks.length;
263                     break;
264                 case 'Home':
265                     nextIndex = 0;
266                     break;
```

The keyboard listener in js/booking.js.

For button clicks, each page uses one Audio object for click.mp3. We pause it and set currentTime to zero before playing again. This restarts the same sound without overlapping copies.

js/booking.js

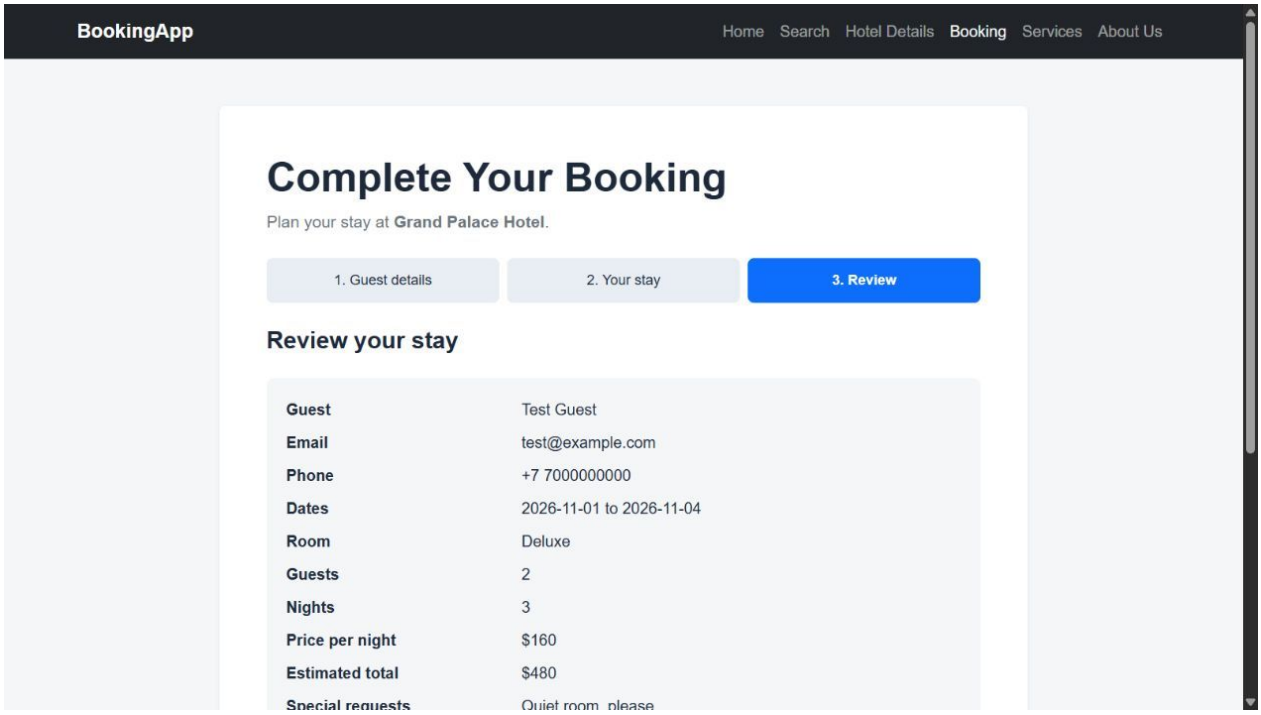
```
278
279     const clickSound = new Audio('image/click.mp3');
280     clickSound.volume = 0.4;
281
282     document.addEventListener('click', function (event) {
283         const button = event.target.closest('button');
284         if (button && !button.disabled) {
285             clickSound.pause();
286             clickSound.currentTime = 0;
287             clickSound.play().catch(function () {
```

The click sound handler in js/booking.js.

4. Booking form

The form has three steps: guest details, stay details and review. Next checks the current fields before moving forward. Back returns to the previous step. We hide the sections instead of removing them, so the input values stay in the form.

changeStep receives showStep as a callback. After changing the current step, it calls showStep to display the correct section.



The booking review with a calculated total.

js/booking.js

```
202
203     function changeStep(direction, onStepChange) {
204         const nextStep = currentStep + direction;
205         if (nextStep < 0 || nextStep >= steps.length) return;
206         if (direction > 0 && !validateStep(currentStep)) return;
207         clearErrors();
208         currentStep = nextStep;
209         onStepChange(currentStep);
210     }
211
212     nextButton.addEventListener('click', function () {
213         changeStep(1, showStep);
214     });
215
216     backButton.addEventListener('click', function () {
217         changeStep(-1, showStep);
218     });
```

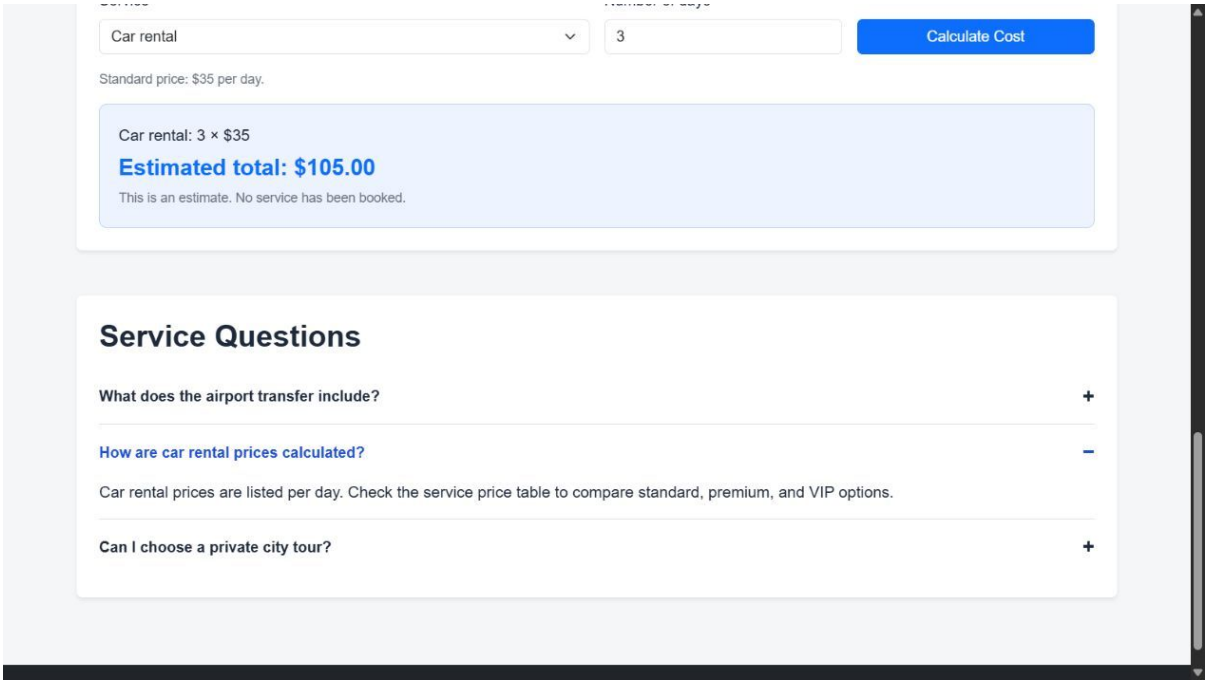
Passing showStep as a callback in js/booking.js.

We fixed the selected hotel by passing its identifier in the URL. getSelectedHotel reads the identifier from the URL and finds the hotel in the shared data. The name and prices now match on the details page, booking form and review.

5. Objects, arrays and calculations

The booking object has methods to calculate the number of nights and the total room price. On Services, the services array stores names and rates. A for loop creates the options in the list.

serviceEstimator uses getService to find the selected service and calculateTotal to multiply its price by the quantity. It uses this to call another method in the same object.



Car rental for three days costs $105.

js/about-services.js

```
52  const serviceEstimator = {
53      getService(serviceId) {
54          return services.find(function (service) {
55              return service.id === serviceId;
56          });
57      },
58
59      calculateTotal(serviceId, quantity) {
60          const service = this.getService(serviceId);
61          return service.price * quantity;
62      }
63  };
64
65  const estimateForm = document.getElementById('service-estimate-form');
```

The serviceEstimator object and its methods.

js/about-services.js

```
76      for (let i = 0; i < services.length; i++) {
77          const option = document.createElement('option');
78          option.value = services[i].id;
79          option.textContent = services[i].name;
80          serviceChoice.appendChild(option);
81      }
```

The loop that creates service options.

We checked that three days of car rental cost $105. Three nights in a Grand Palace deluxe room cost $480.

6. Animation, checks and deployment

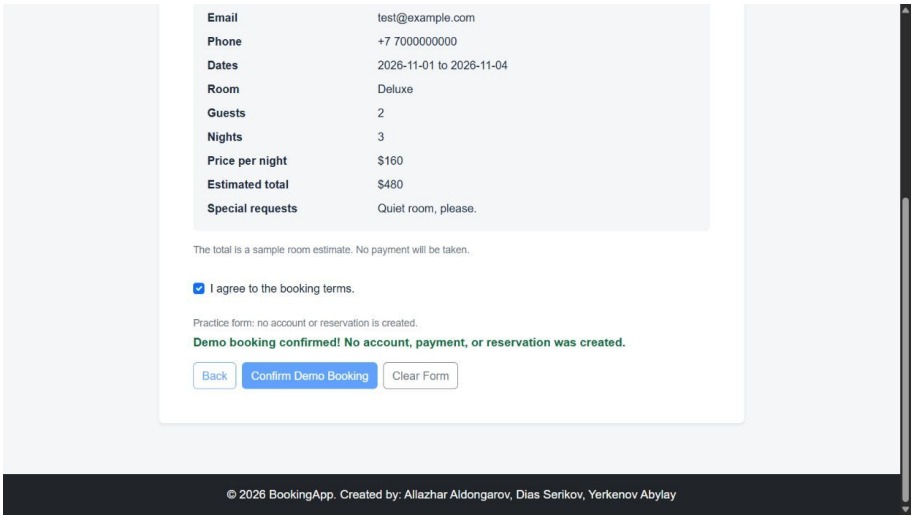
The booking steps animate when Next or Back is clicked. The service result also animates after Calculate Cost. JavaScript adds a class, and the animation rules are in CSS.

```
js/about-services.js
104  estimateResult.classList.add('estimate-reveal');
105  });
106
107  estimateResult.addEventListener('animationend', function () {

css/ablstyle.css
126  .estimate-reveal {
127      animation: estimate-reveal 0.35s ease;
128  }
134  @keyframes estimate-reveal {
135      from {
136          opacity: 0;
137          transform: translateY(10px);
138      }
139
140      to {
141          opacity: 1;
142          transform: translateY(0);
143      }
144  }
```

JavaScript starts the animation defined in CSS.

HTML, CSS and JavaScript are in separate files. We checked the filters, Read More, keyboard navigation, validation, Back and reset. The forms are demos: they do not create real bookings or send messages.



Confirmation after entering valid booking details.

We pushed the project to GitHub and deployed it:
Website: https://aishhko.vercel.app/
Repository: https://github.com/jjestar/web
GitHub Pages: https://jjestar.github.io/web/

The main issue was keeping the selected hotel the same between pages. Shared data and the identifier in the URL solved it. We learned how events, callbacks and DOM updates work together.